Reviewer Pack — Minimal Topological Entropy Bounds Circuit Depth

Entry d0022a080923 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 09:07:08 UTC

Minimal Topological Entropy Bounds Circuit Depth

Entry ID: `d0022a080923` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 09:07:08 UTC

1. Conjecture

Field A (mathematical branch): Topological Dynamics **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> For every Boolean circuit C with depth d and size s, the minimal topological entropy H(C) of C satisfies H(C) = Θ(log(s)). Furthermore, for any constant c > 0, there exists a polynomial-time computable function f(n) such that for all circuits C with depth d and size s ≤ n, if |C| ≤ f(d), then H(C) ≤ cd.

Rationale (proposer's reasoning):

> Topological dynamics has been used to study the complexity of iterated maps, which can be seen as a simplified model of computation. By extending this approach to Boolean circuits, we may uncover new insights into the structure and complexity of circuits. The topological entropy is a measure of the complexity of dynamical systems and could potentially reveal hidden structures in circuits that are not apparent through traditional complexity measures.

Taxonomy category: `Topological_Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5fd48fc5a0e71cd5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all circuits C with depth d and size s ≤ n, |C| ≤ f(d) implies H(C) ≤ cd and the correlation coefficient between entropy and size is ≥ 0.8 with p-value < 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - topological dynamics AND boolean circuit complexity AND minimal topological entropy - circuit depth IN BOOLEAN MODE AND topological entropy θ(log(size)) - Boolean circuit complexity polynomial-time computable function f(n) AND topological entropy ≤ cd

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(depth, size):
        if depth == 1:
            return [random.choice([0, 1])]
        else:
            subcircuits = []
            for _ in range(size):
                subcircuit = generate_circuit(depth - 1, size // depth)
                subcircuits.append(subcircuit)
            return subcircuits

    def compute_entropy(circuit):
        if isinstance(circuit, list):
            entropy = 0
            for subcircuit in circuit:
                entropy += compute_entropy(subcircuit)
            return entropy / len(circuit)
        else:
            return 1

    n_max = 40
    instances_tested = 0
    metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
```

```
        for _ in range(5):
            depth = random.randint(1, n)
            size = random.randint(1, n)
            circuit = generate_circuit(depth, size)
            entropy = compute_entropy(circuit)
            metric_value += entropy
            instances_tested += 1

            if entropy < math.log(size):
                conjecture_holds = False
                counterexample = f"Circuit with depth {depth}, size {size} has entropy {entropy} <

    mean_metric_value = metric_value / instances_tested
    support_fraction = instances_tested / (6 * 5)

    return {
        "metric_name": "Minimal Topological Entropy",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad96a9fc.py", line 76, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad96a9fc.py", line 51, in run_trial
    entropy = compute_entropy(circuit)
              ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad96a9fc.py", line 35, in compute_entro
    entropy += compute_entropy(subcircuit)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad96a9fc.py", line 35, in compute_entro
    entropy += compute_entropy(subcircuit)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad96a9fc.py", line 36, in compute_entro
    return entropy / len(circuit)
           ~~~~~~~~^~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces the required data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20774
2	preregistration	ollama_remote	glm4:latest	0	0	11870
3	novelty	ollama_remote	glm4:latest	0	0	15734
4	novelty	ollama_remote	glm4:latest	0	0	13450
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12647
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14406
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10890
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21363
9	judge	ollama_remote	glm4:latest	0	0	9113

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 130244 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d0022a080923.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d0022a080923.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d0022a080923.tar.gz` (if generated)